神经网络

回顾：线性分类器

图片

包含**32x32x3**数字的数组
(总共3072个数字)

→ f(**x**,**W**) → **10个数字表示类别得分。**

↑

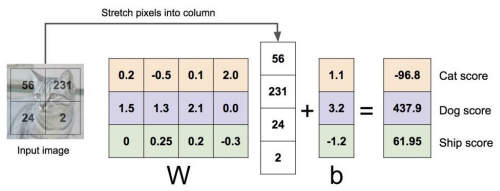
W

参数或权重

$$f(x,W) = Wx + b$$

代数视角

f(x,W) = Wx

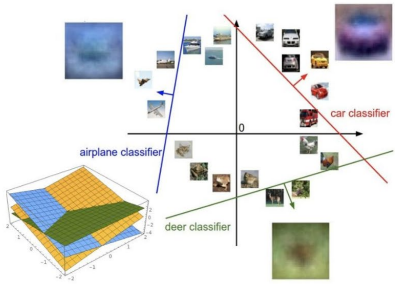


可视化视角

One template per class

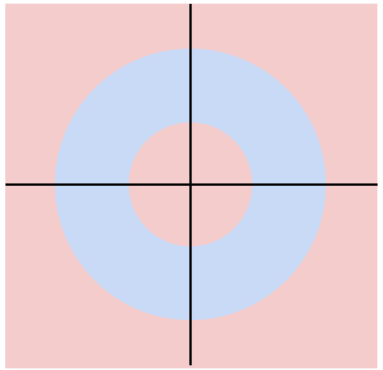
几何视角

Hyperplanes cutting up space



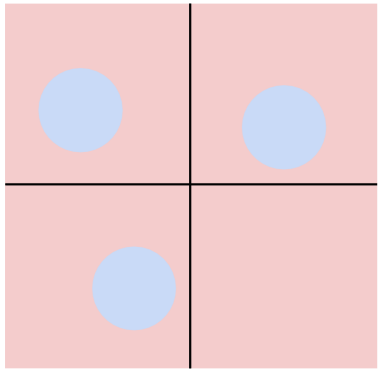
Class 1:
1 <= L2 norm <= 2

Class 2:
其他



Class 1:
三个圆

Class 2:
其他

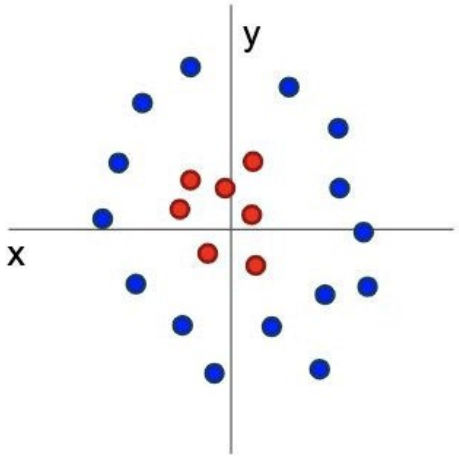


问题:线性分类器不够强大

视觉视角

线性分类器每个类学习
一个模板

几何视角

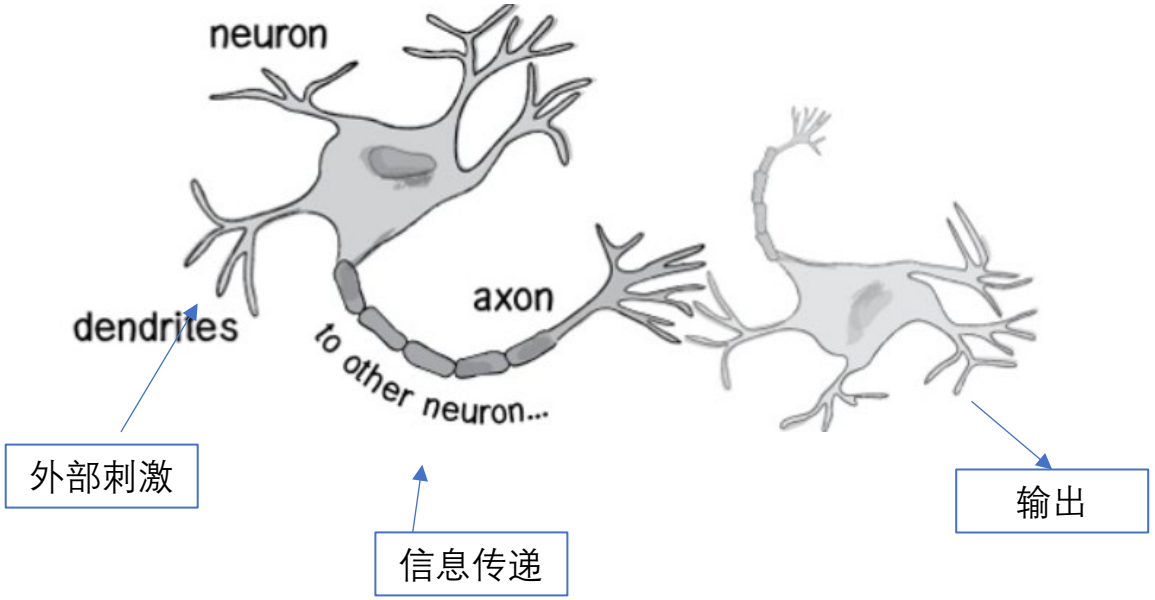


线性分类器只能绘
制线性决策边界

https://playground.tensorflow.org/

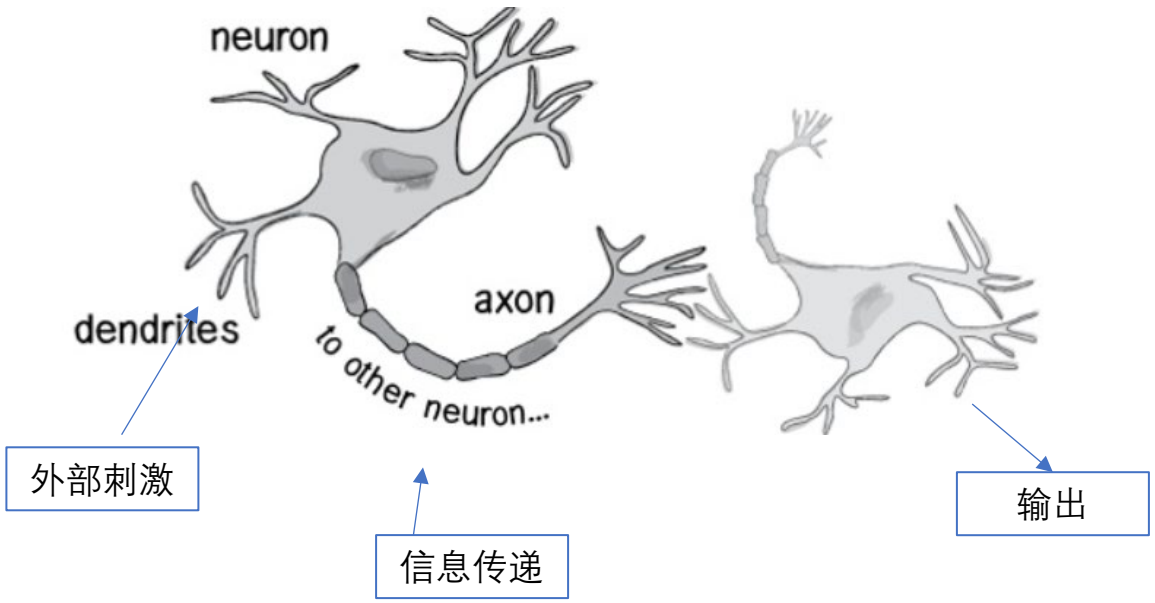
什么是神经网络?

生物神经网络的基本工作原理:
一个神经元的输入端有多个树突，主要是用来接收输入信息的。输入信息经过突触处理，将输入的信息累加，当处理后的输入信息大于某一个特定的阈值，就会把信息通过轴突传输出去，**这时称神经元被激活**。相反，当处理后的输入信息小于阈值时，神经元就处于抑制状态，它不会像其他神经元传递信息。或者传递很小的信息。



什么是神经网络?

人是怎么思考的? --生物神经网络



感知器:

1, 外部刺激通过神经末梢, 转化为电信号, 转导到神经细胞（又叫神经元）

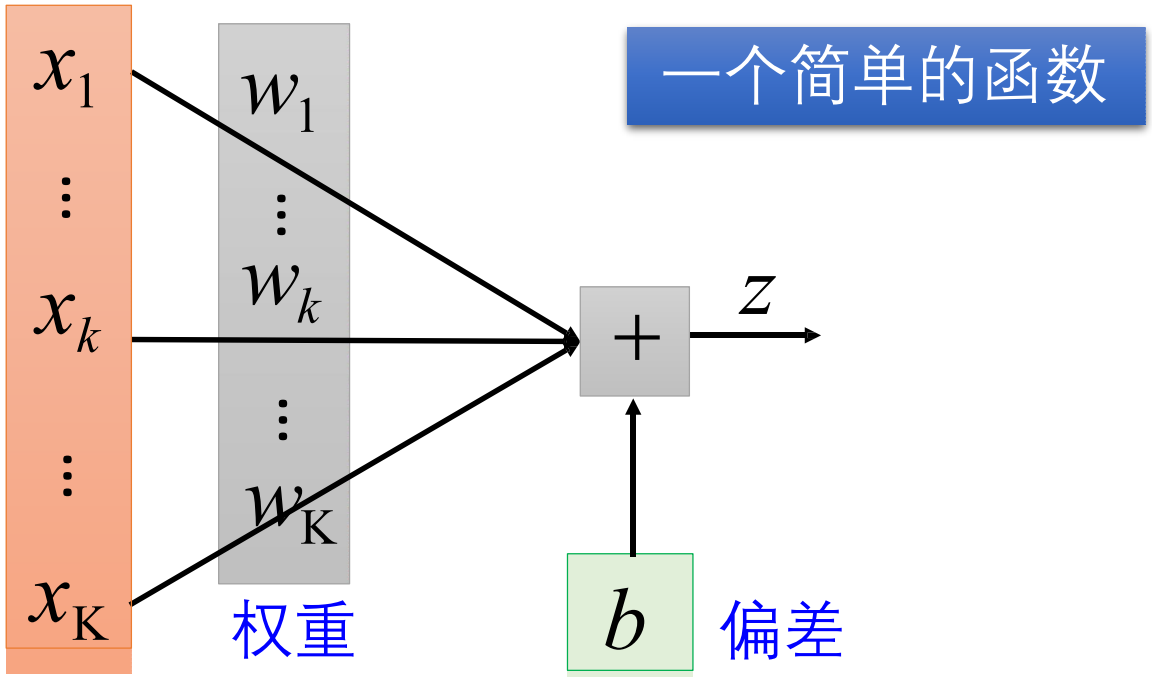
2, 无数神经元构成神经中枢

3, 神经中枢综合各种信号, 做出判断

4, 人体根据神经中枢的指令, 对外部刺激做出反应

线性分类器

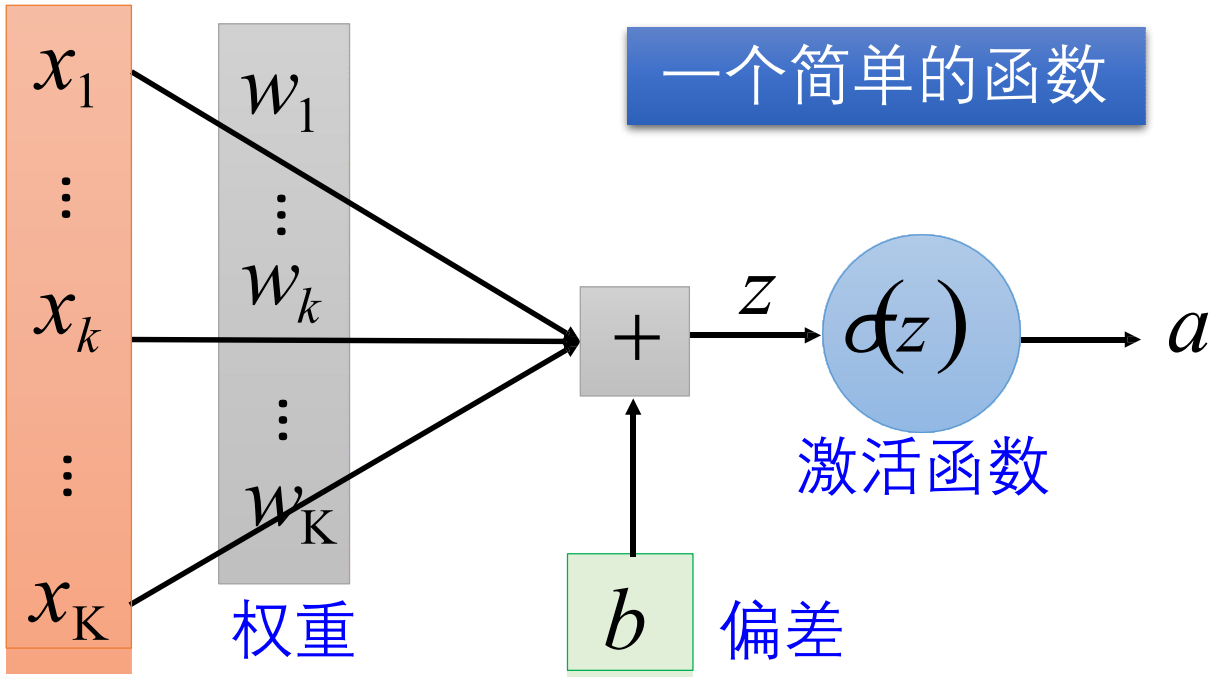
$$S=f(x;w) = x_1 w_1 + \cdots + x_k w_k + \cdots + x_K w_K + b$$



神经网络

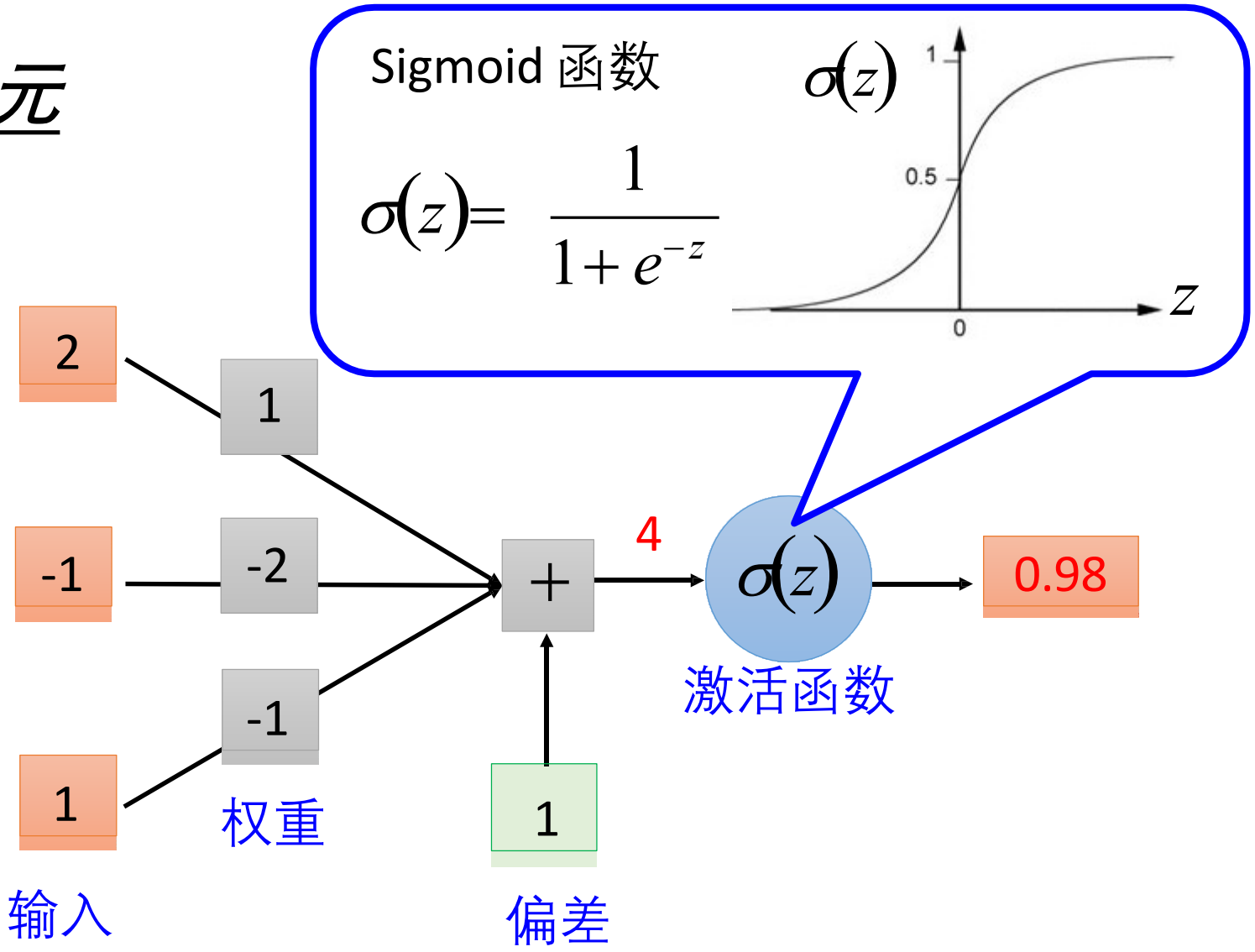
神经元

$$z = x_1 w_1 + \cdots + x_k w_k + \cdots + x_K w_K + b$$



神经网络

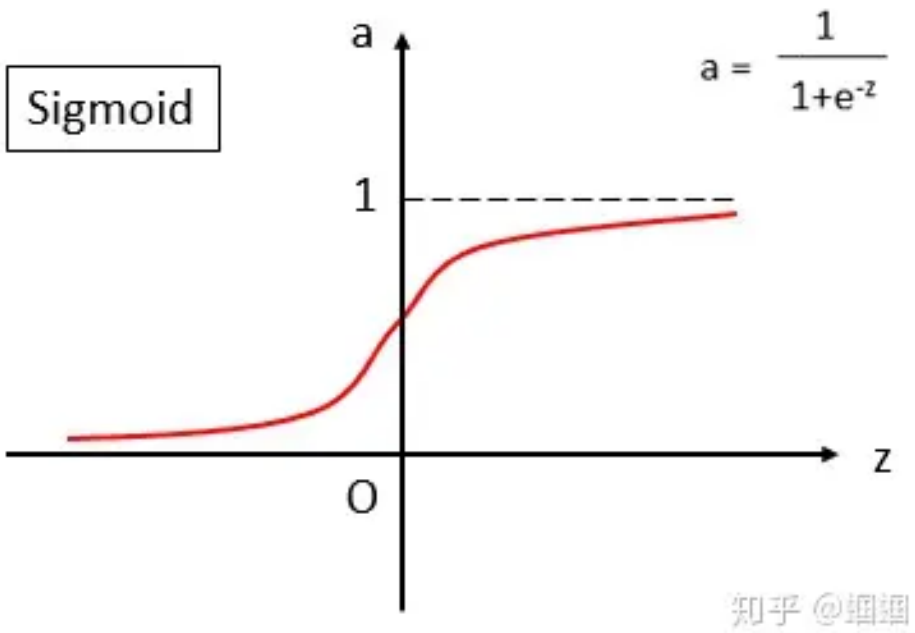
神经元


Sigmoid 函数
$\sigma(z) = \frac{1}{1+e^{-z}}$
$\sigma(z)$
1
0.5
0
z
2
-1
1
1
-2
-1
权重
+
4
$\sigma(z)$
激活函数
0.98
1
偏差
输入

激活函数

Desmos | 图形计算器

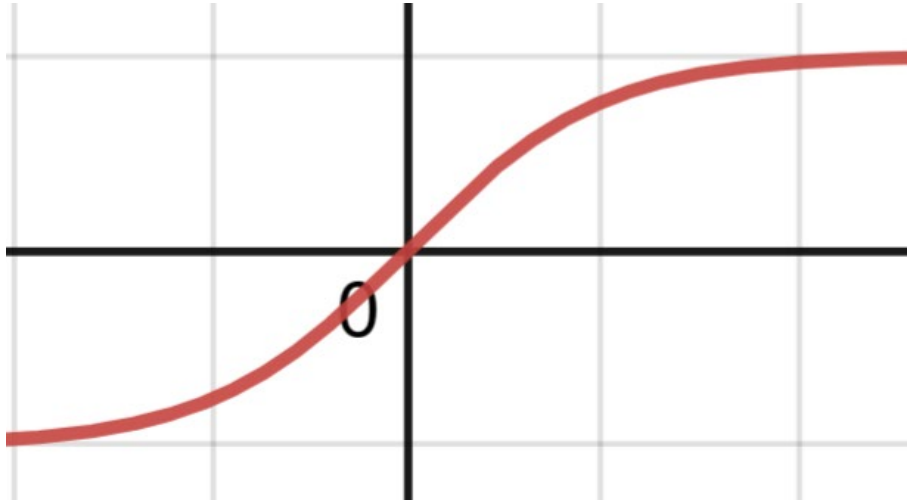
- Sigmoid 函数
  - 输出范围在 0 到 1 之间
  - sigmoid 函数常用于输出层的二元分类问题



激活函数

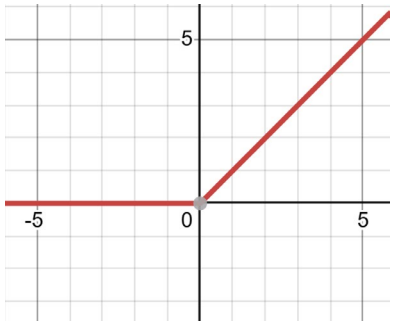
- **tanh函数**
  - 输出范围在 -1 到 1 之间

$$a = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



[Desmos | 图形计算器]

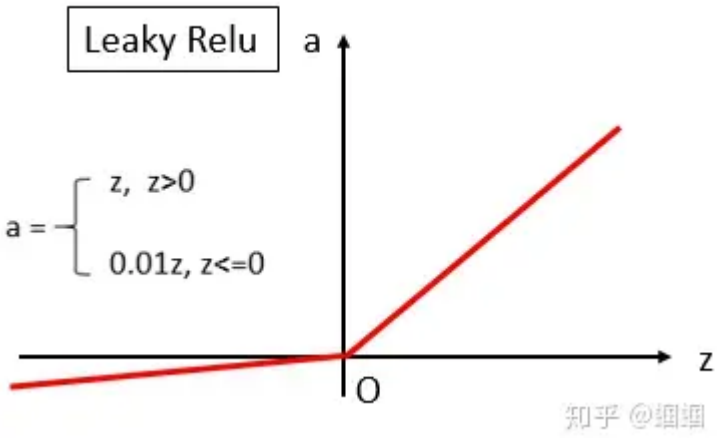
激活函数



- **ReLU函数**
  - ReLU的全称是Rectified linear unit（线性整流单元）
  - a=max(z,0)
  - 输出为正输入时与输入相等，否则为 0。
  - 计算简单，有效缓解了梯度消失问题。

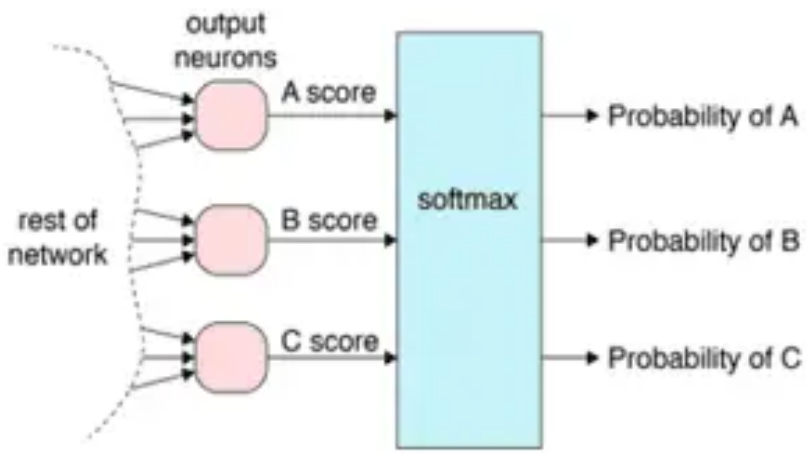
激活函数

- **Leaky ReLU函数**
  - ReLU的变种，避免ReLU在输入小于0时输出为0



激活函数

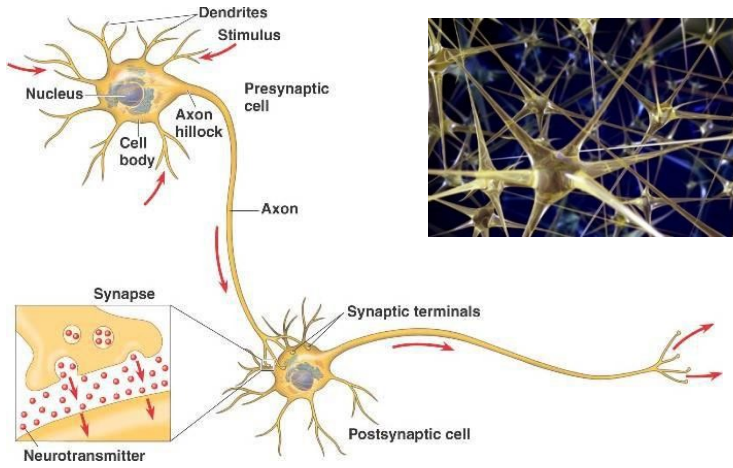
- **softmax函数**
  - 用于多类分类问题，将输入转换为类别概率分布。
  - 输出是正数，且总和为 1。
  - 通常用于输出层。

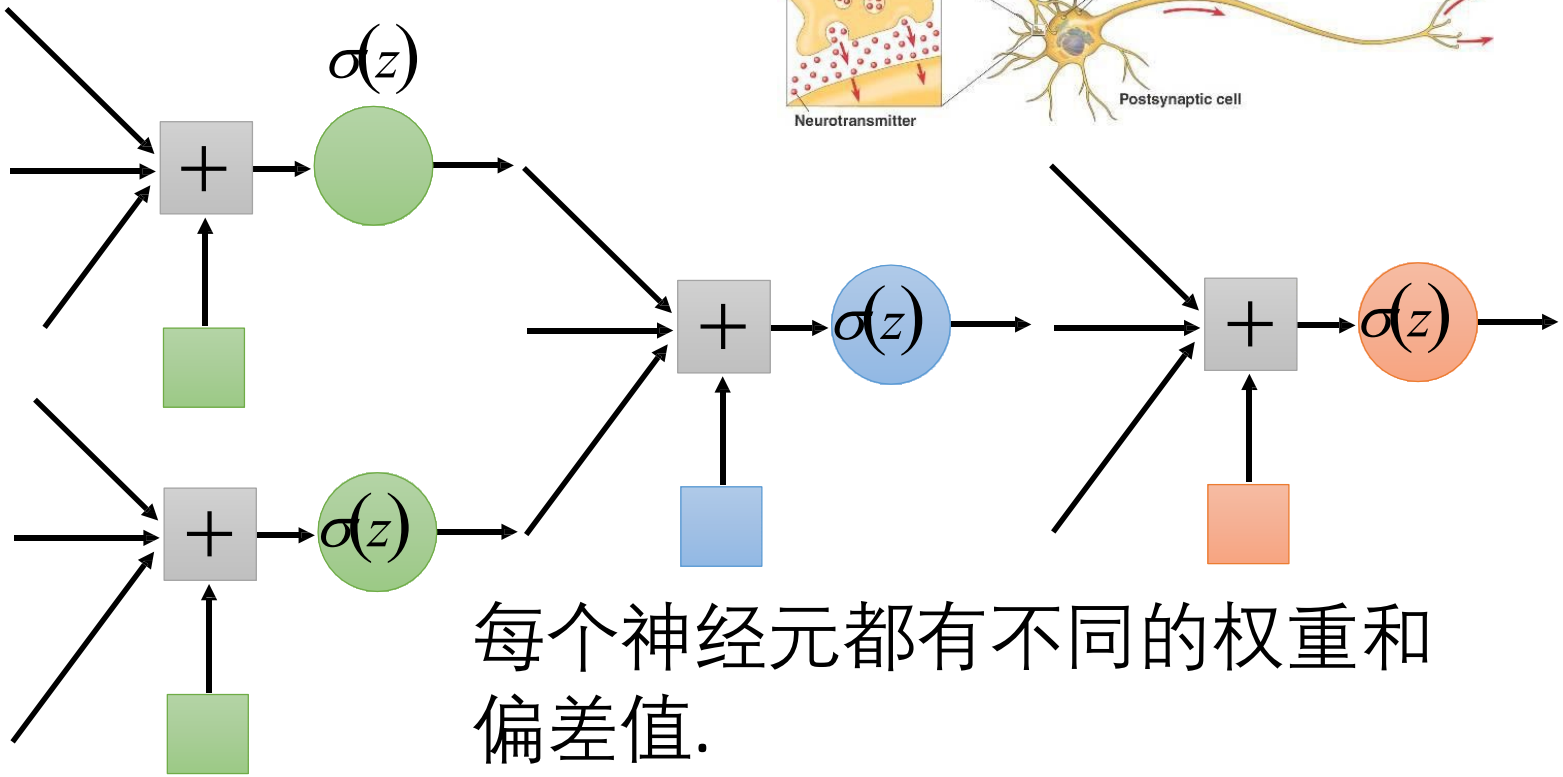


$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

神经网络

不同的连接导致不同的网络结构

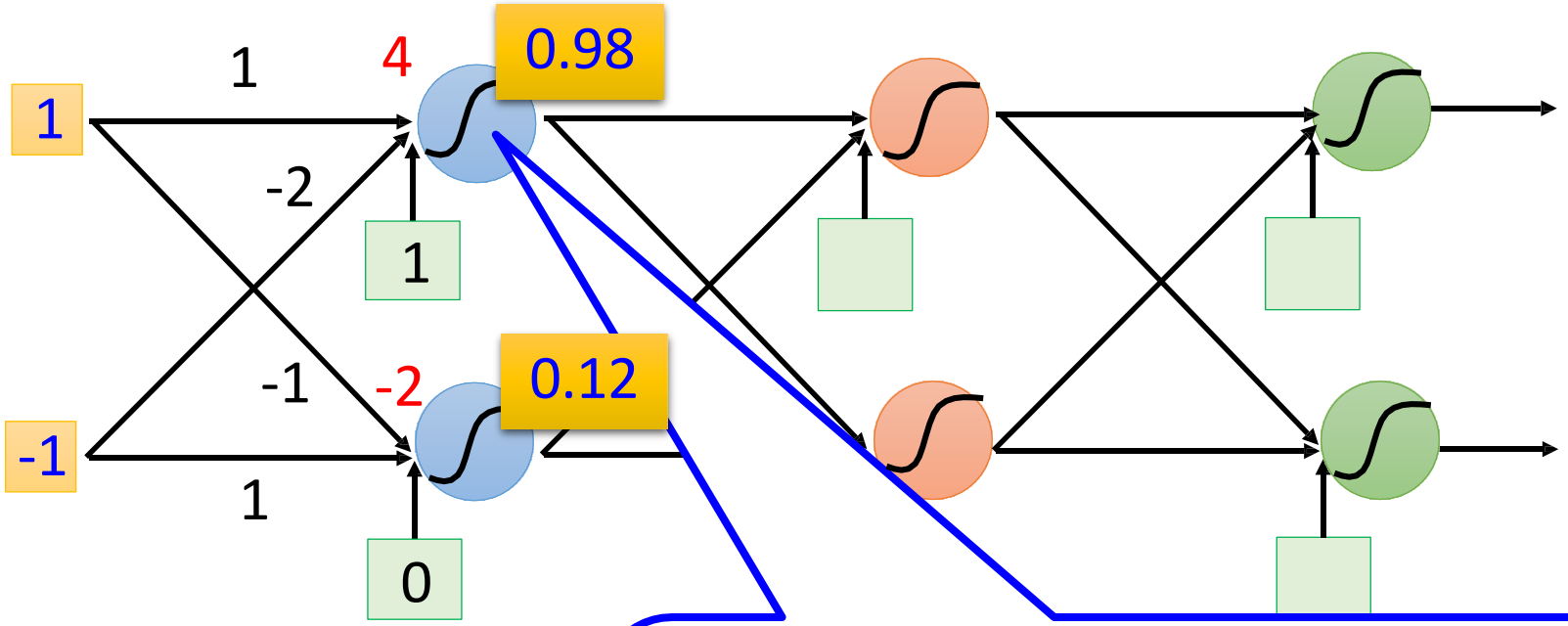




每个神经元都有不同的权重和偏差值.

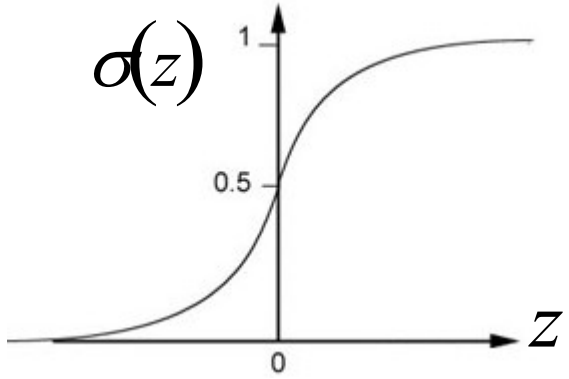
权重和偏差都是神经网络的参数 θ

全连接神经网络

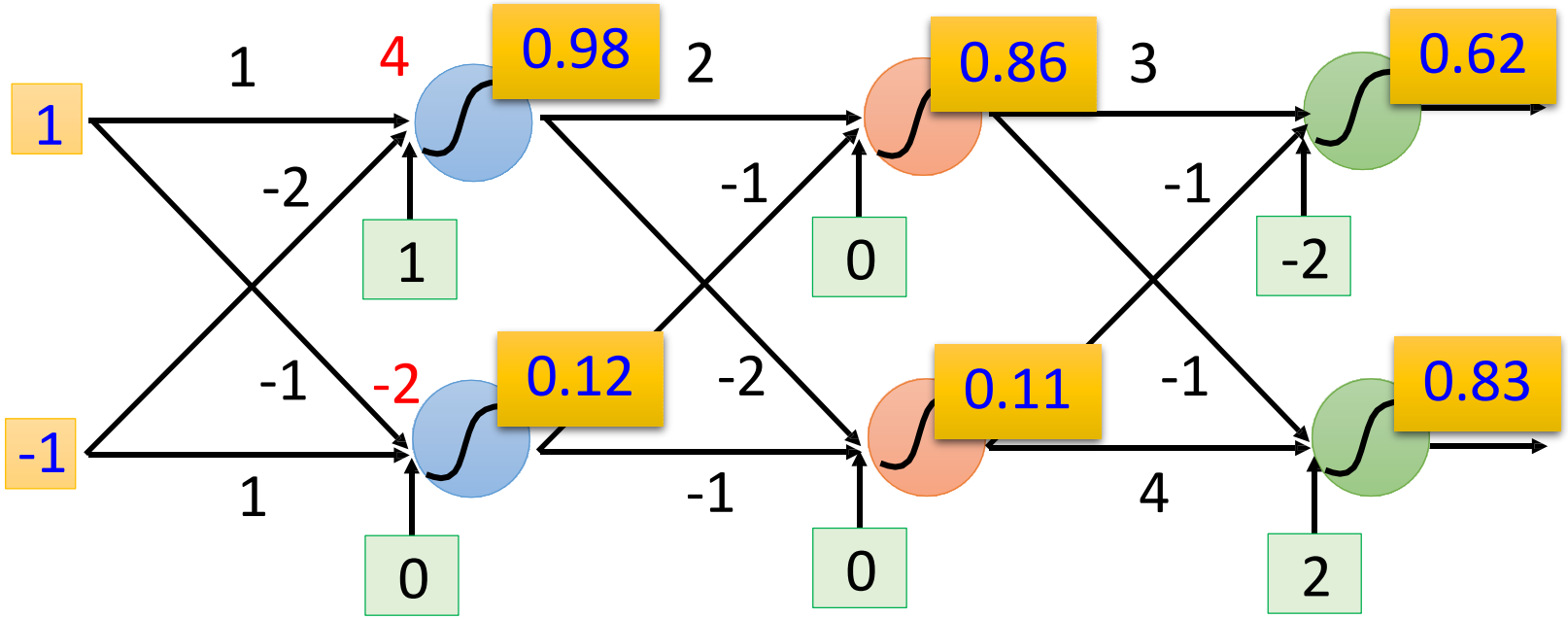


Sigmoid 函数

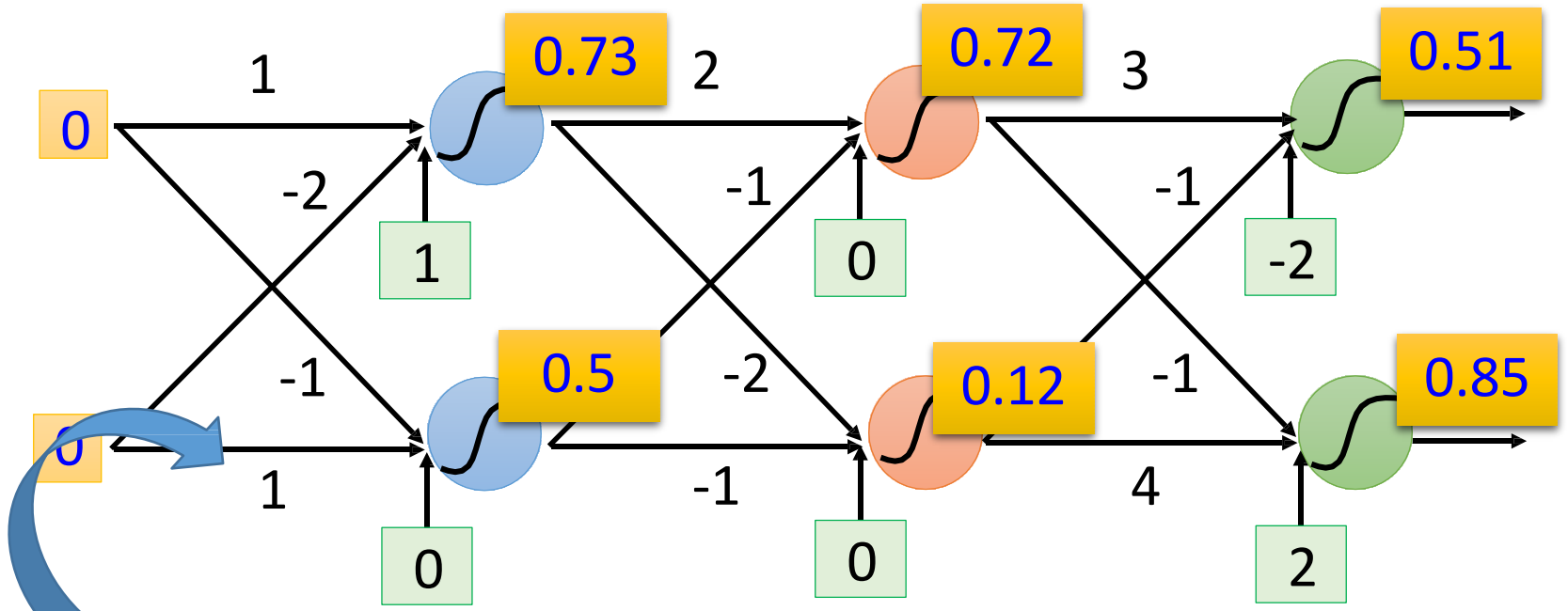
$$\sigma(z) = \frac{1}{1+e^{-z}}$$



全连接神经网络



全连接神经网络



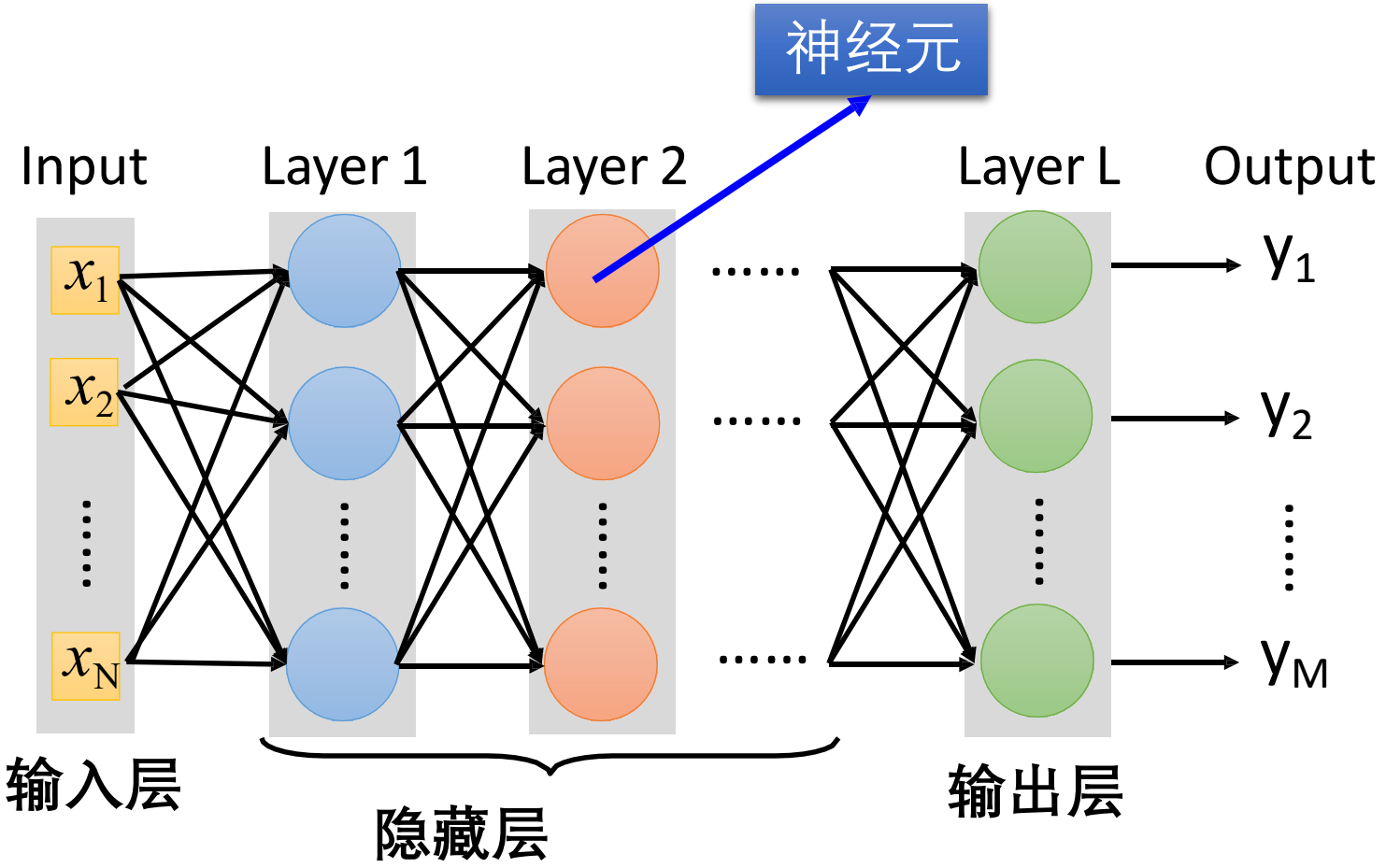
这是一个函数.
输入向量, 输出向量

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix} \quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

给定参数 θ, 定义一个函数

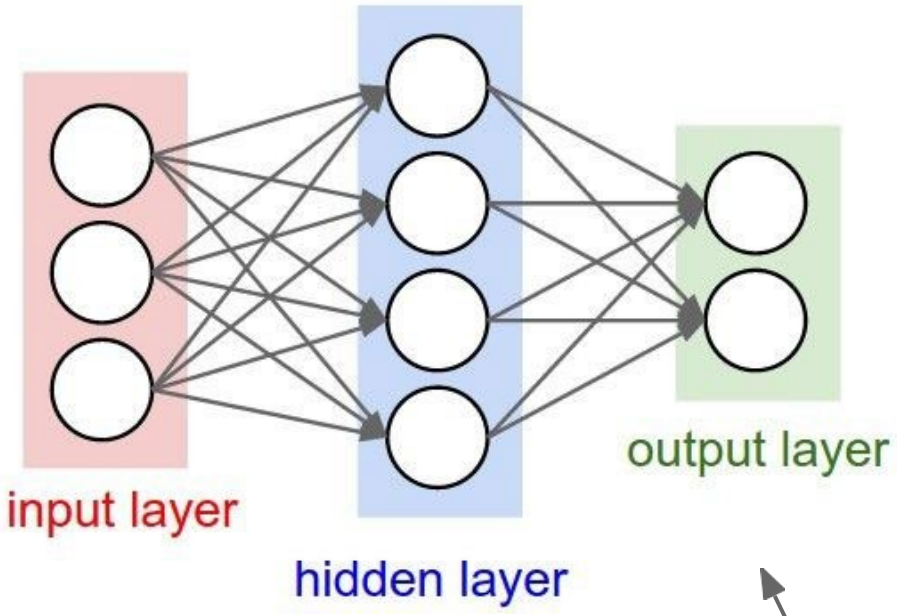
给定网络结构，定义一个函数集

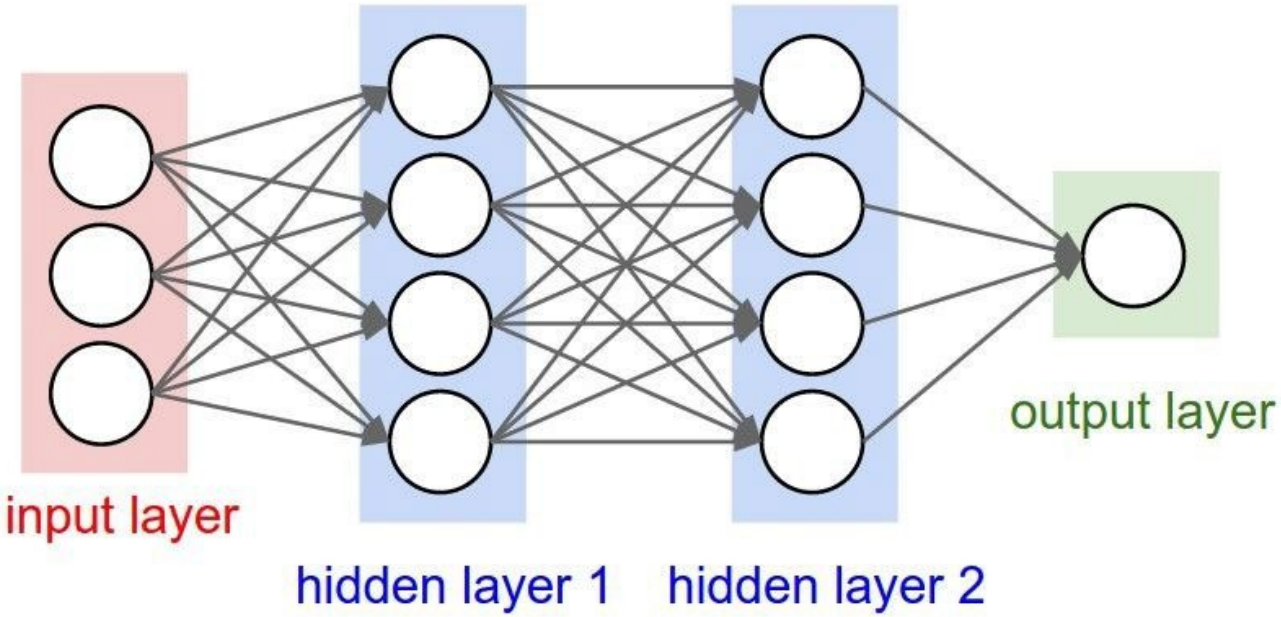
全连接神经网络



深度意味着很多隐藏层

全连接神经网络



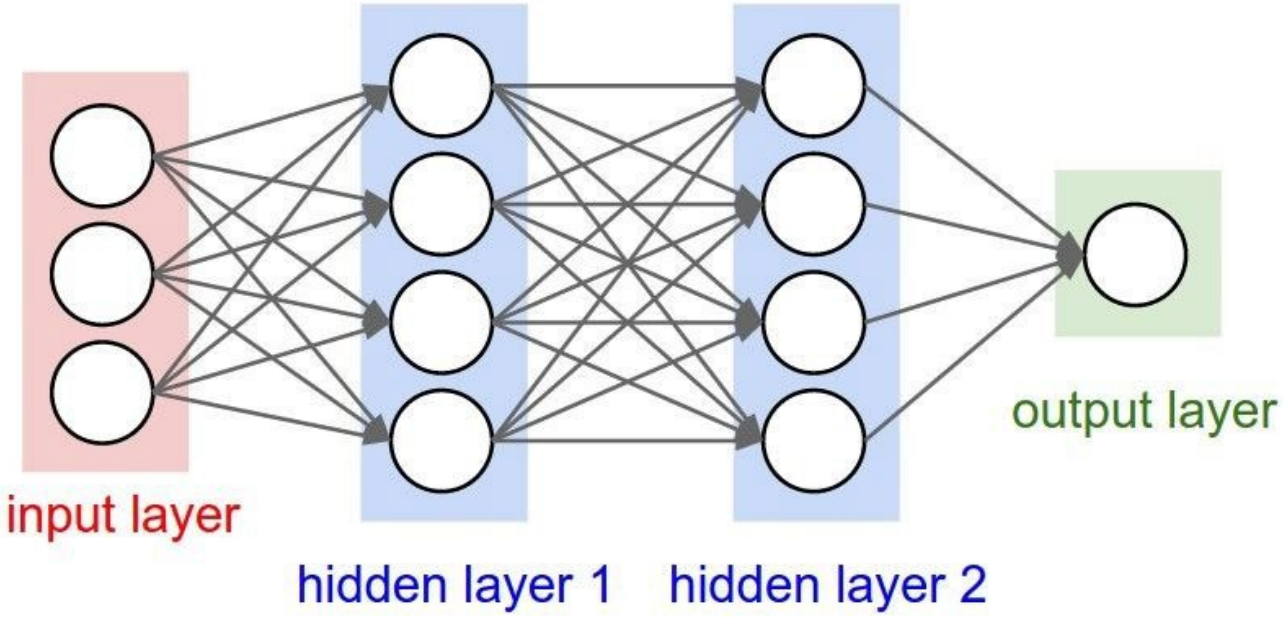


“2层神经网络”, 或
“1层隐藏层神经网络”

“全连接” 层

“3层神经网络”, 或
“2层隐藏层神经网络”

示例神经网络的前馈计算

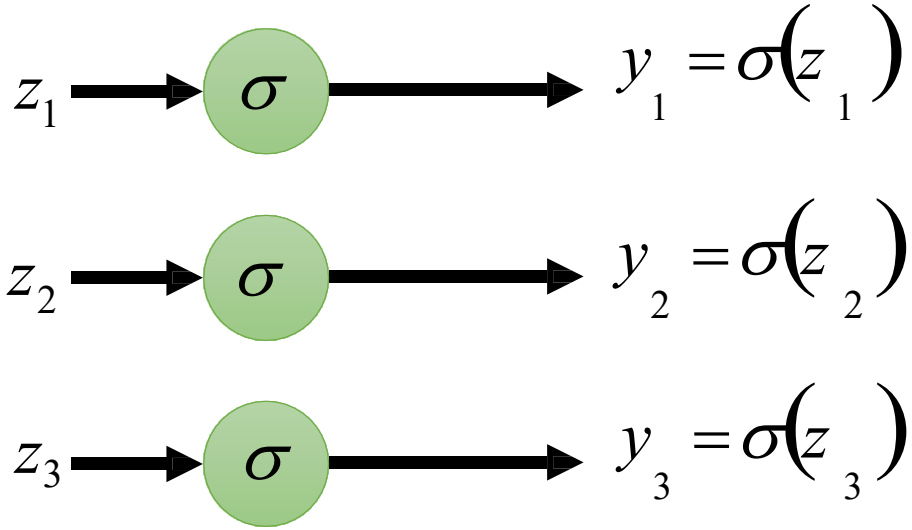



```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

输出层(Option)

- Softmax 层作为输出层

普通层



一般来说，网络的输出可以是任何值.

可能不容易解释

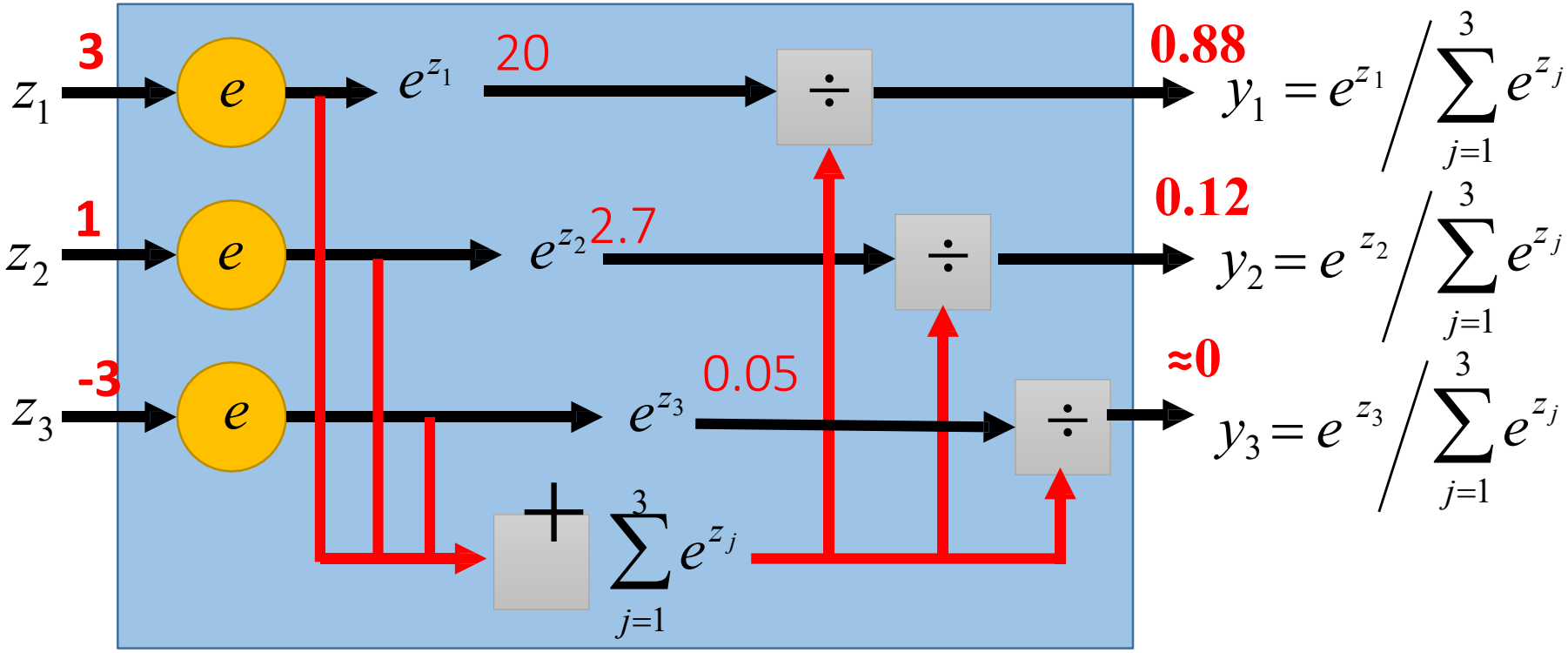
输出层(Option)

- Softmax 层作为一个输出层

Probability:

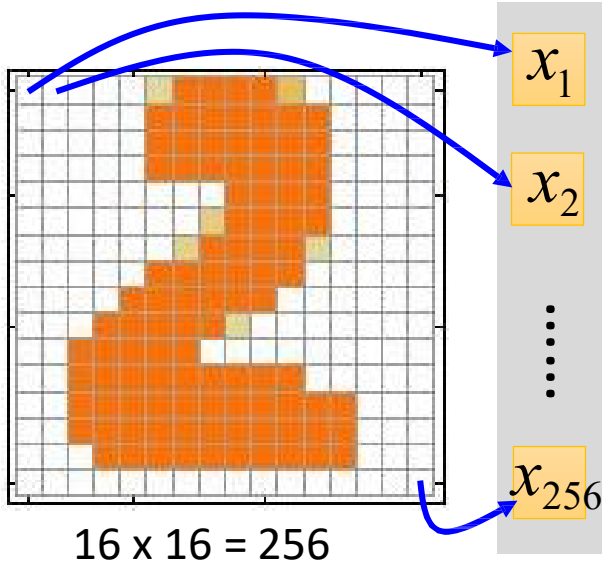
- $1 > y_i > 0$
- $\sum_i y_i = 1$

Softmax Layer



示例

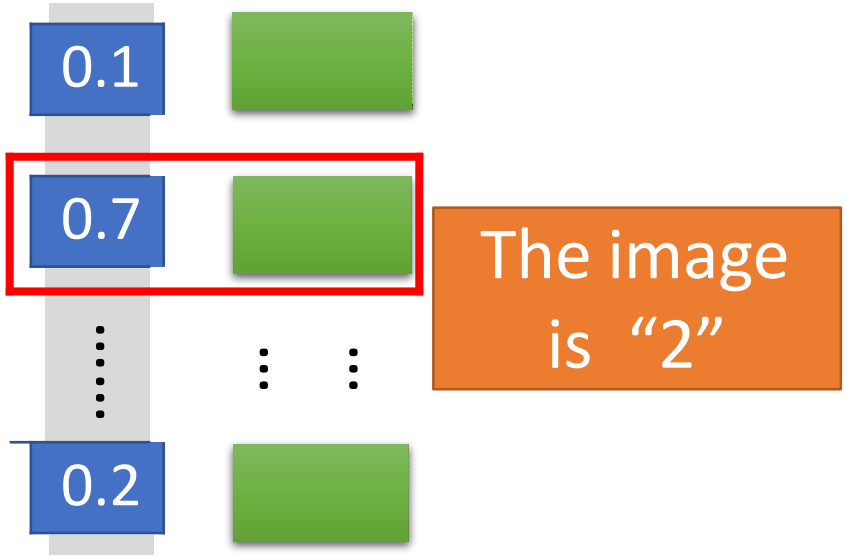
输入



Ink → 1

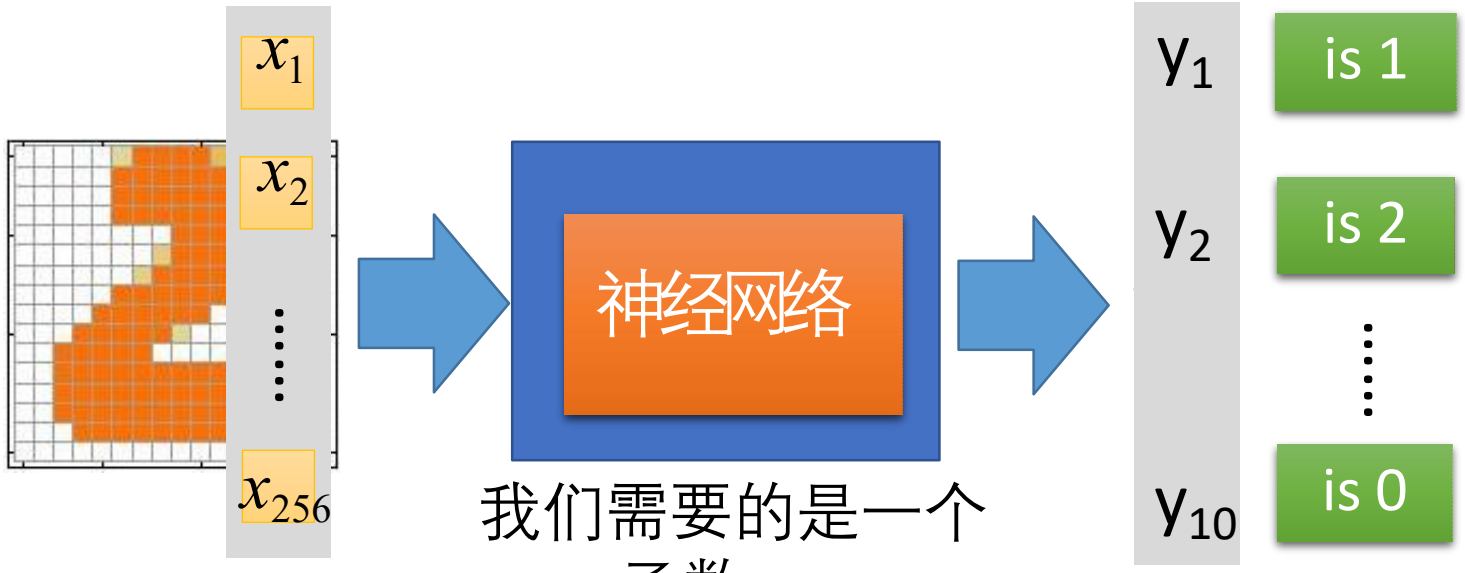
No ink → 0

输出



示例

- 书写体数字识别

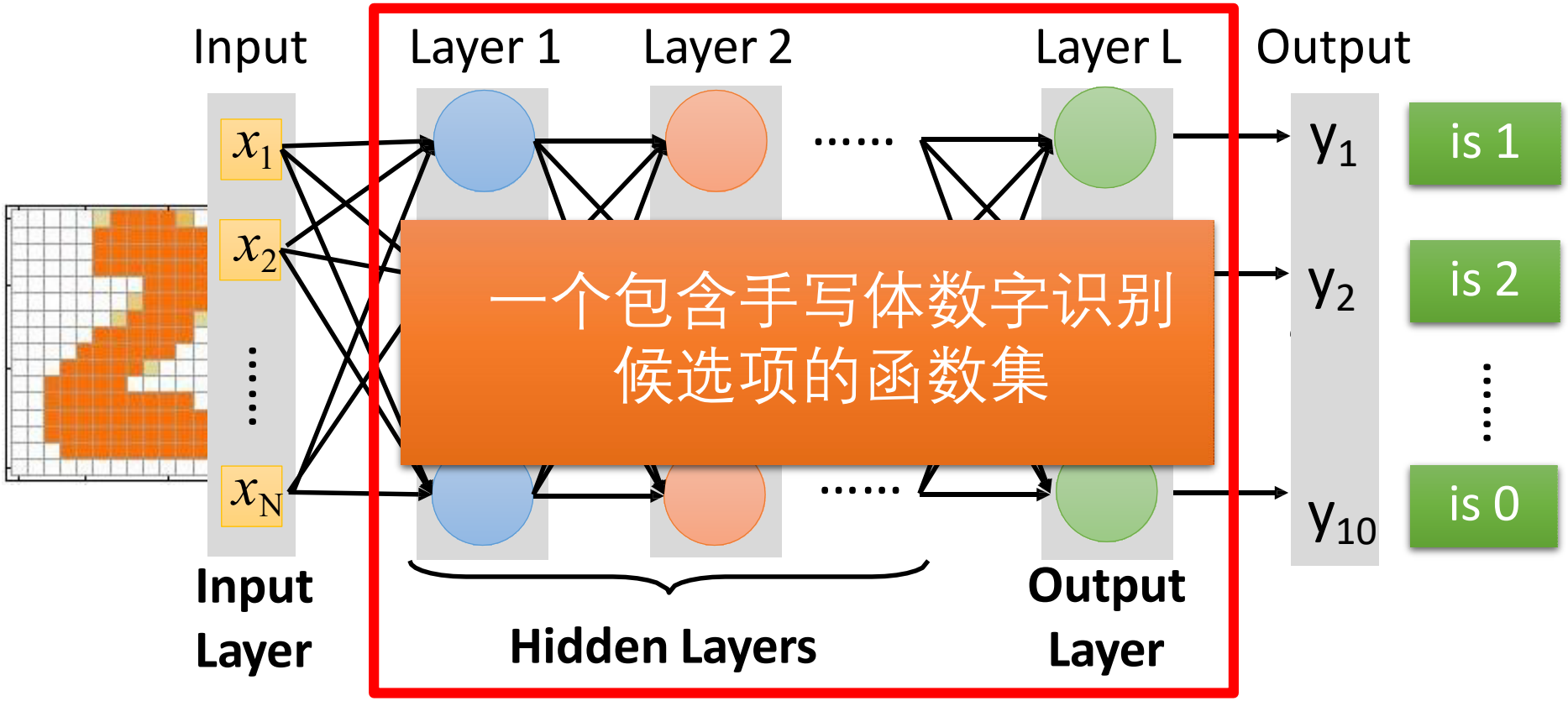


我们需要的是一个函数 ……

Input:
256-dim vector

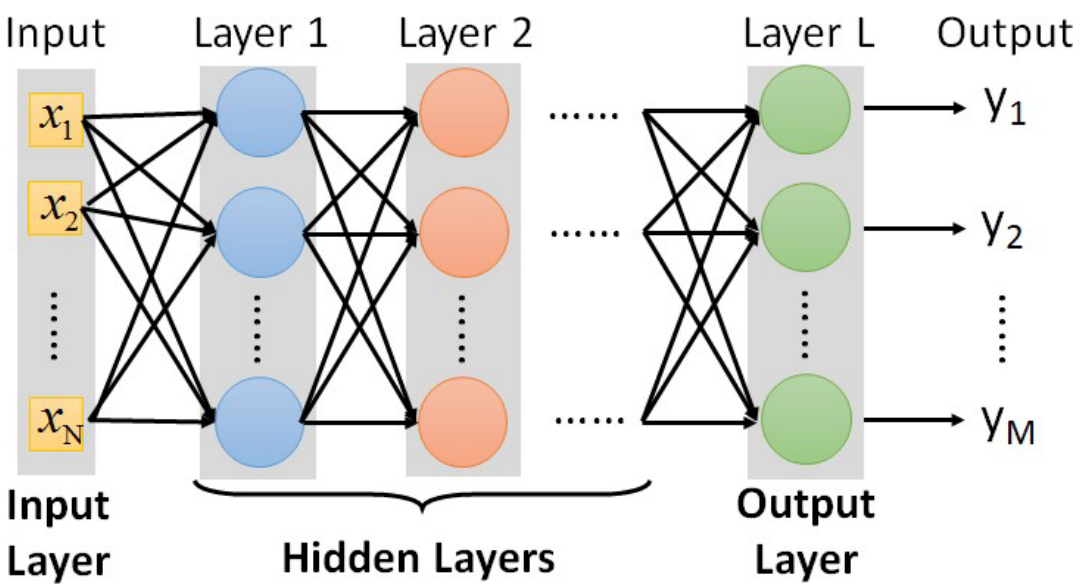
output:
10-dim vector

示例



你需要决定网络结构，让你的函数有好的功能

FAQ



- Q: 多少层?每层有多少个神经元?

试验和错误 + 直觉

- Q: 网络结构是否可以自动确定?

训练数据

- 准备训练数据:图片和他们的标签

“5”

“0”

“4”

“1”

“9”

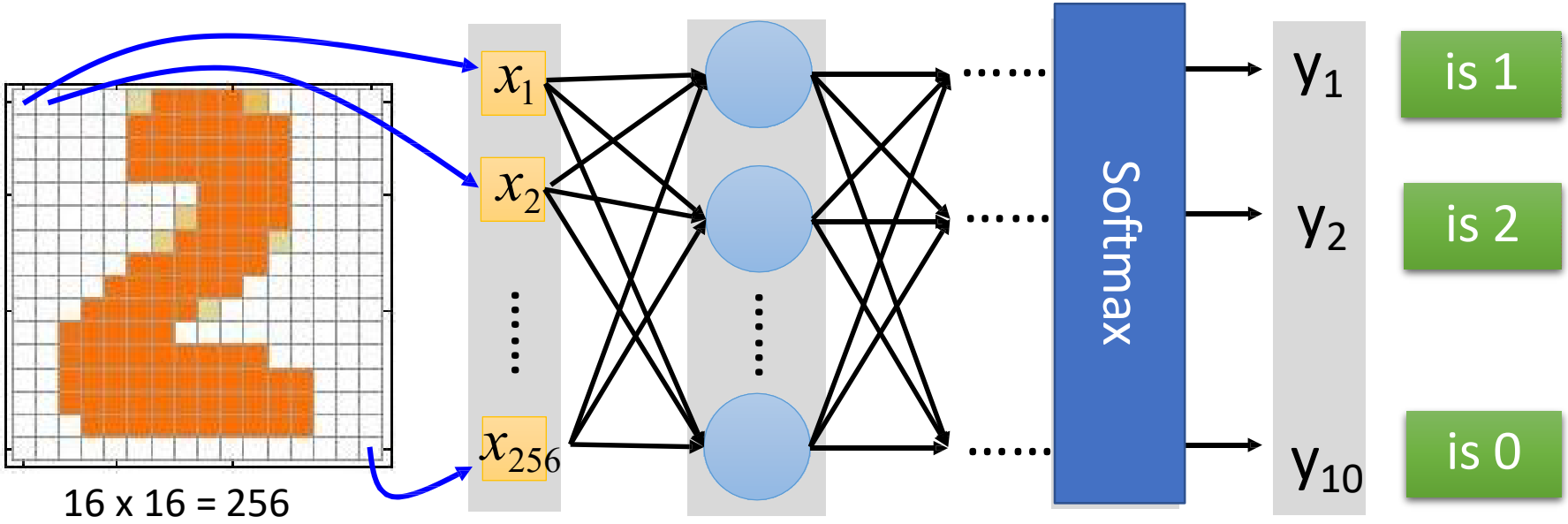
“2”

“1”

“3”

学习目标是在训练数据上定义的

学习目标



Ink → 1

No ink → 0

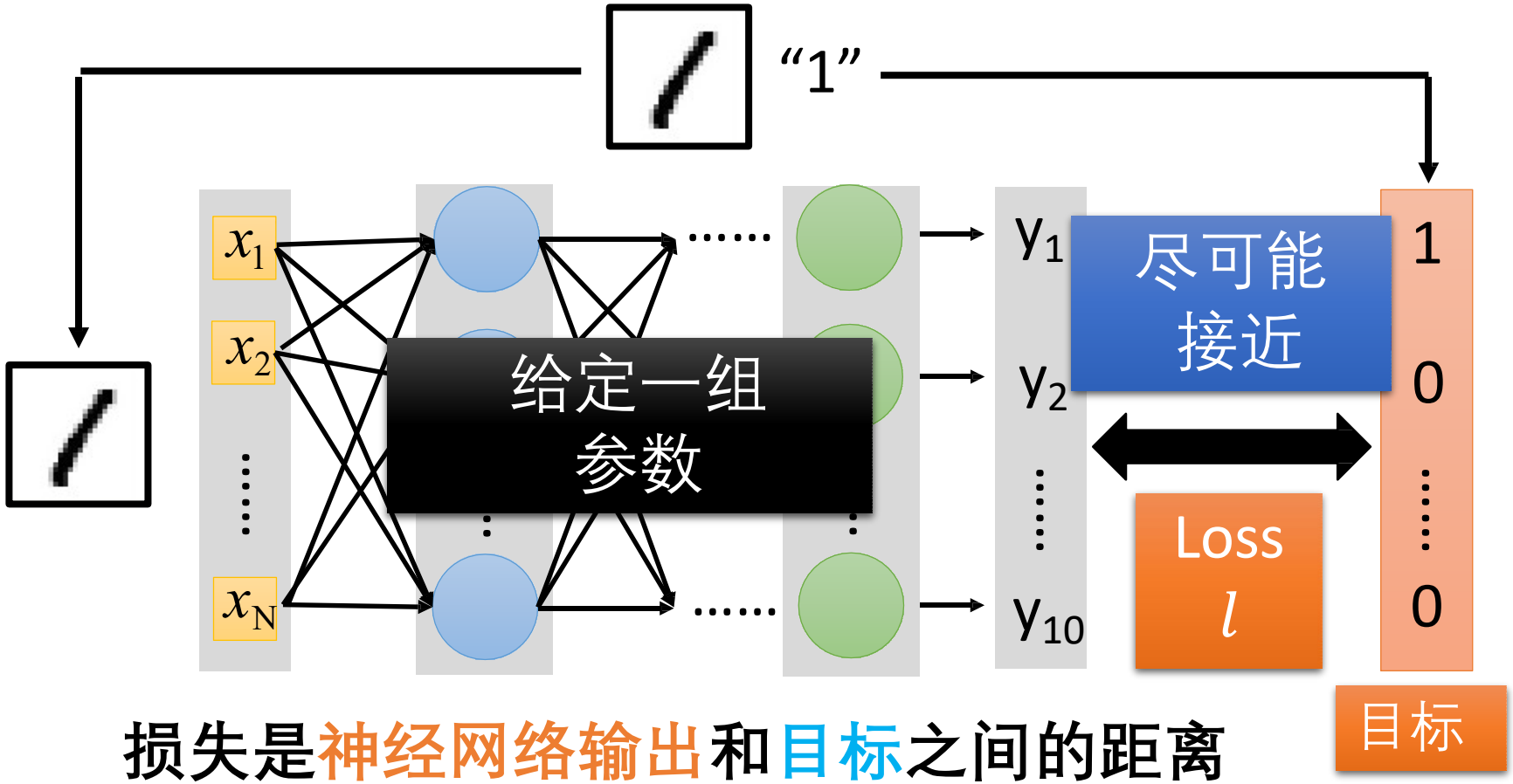
学习目标是 ……

Input: → y_1 是最大的

Input: → y_2 是最大的

损失

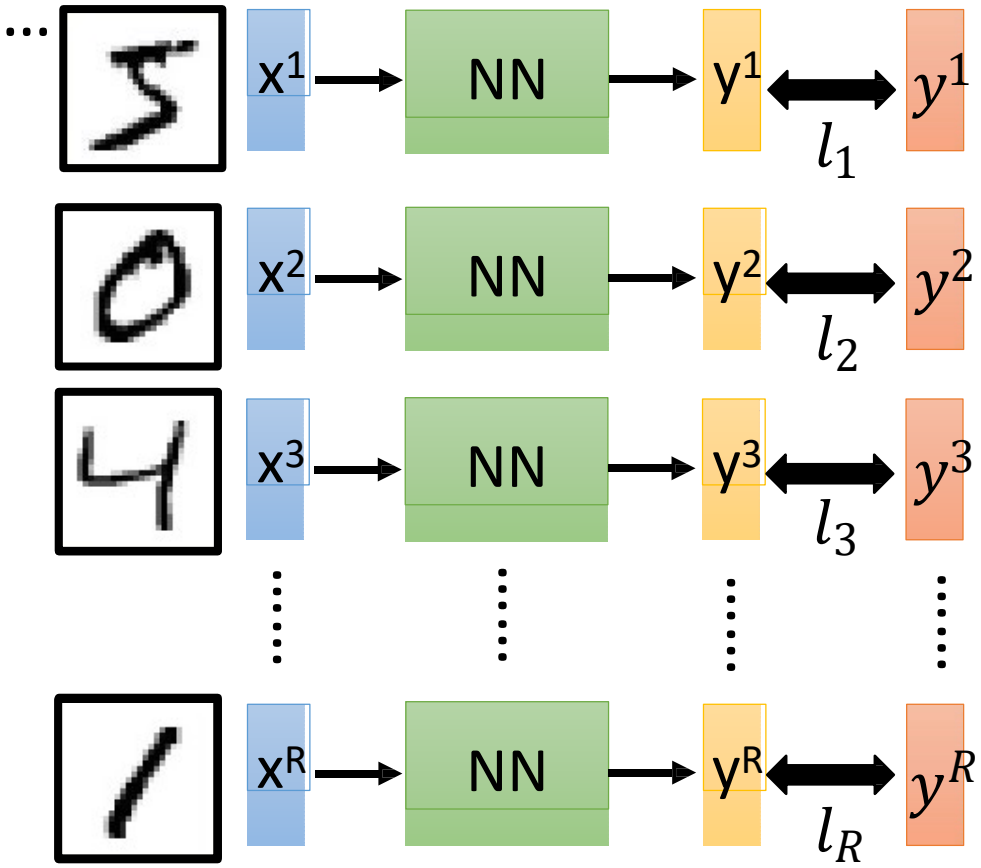
一个好的函数尽可能使损失最低



损失是神经网络输出和目标之间的距离

全部损失

对**所有的训练数据**



全部损失:

$$L = \sum_{r=1}^{R} l_r$$

尽可能的小

找到一个函数使损失函数 L 最小

找到参数 $\boldsymbol{\theta}$ 使损失函数 L 最小